

Universidad Autónoma de Campeche

Ingeniería en Tecnología de Software

Materia

Concurrencia y Paralelismo

Maestro

Sergio A Noh Puch

Práctica

Medición de tiempo de ejecución con flujos con y sin buffer

Alumna:

Gladys Candy Chumá Pérez

Matrícula

70238

Introducción

En esta práctica trabajamos con diferentes flujos de lectura y escritura de archivos en Java, utilizando métodos con buffer y sin buffer. Realizamos varias pruebas con distintas cantidades de datos para medir cuánto tardaba cada uno y poder comparar sus resultados.

Objetivo

Comparar los tiempos de lectura y escritura de archivos de texto y binarios, utilizando flujos con buffer y sin buffer, para observar las diferencias en su rendimiento al trabajar con distintas cantidades de datos.

Tablas de resultados

Prueba con 100,000 elementos

Flujo utilizado	**Tamaño (elementos)**	**Ejec. 1 (ms)**	**Ejec. 2 (ms)**	**Ejec. 3 (ms)**	**Promedio (ms)**
FileWriter	100,000	110.908 ms	128.042 ms	85.678 ms	108.209 ms
BufferedWriter	100,000	62.734 ms	69.390 ms	51.263 ms	61.129 ms
FileReader	100,000	221.242 ms	213.323 ms	164.277 ms	199.614 ms
BufferedReader	100,000	30.050 ms	30.335 ms	44.257 ms	34.881 ms
FileOutputStream	1,000,000	596.361 ms	578.909 ms	579.554 ms	584.941 ms
BufferedOutputStream	1,000,000	5.561 ms	5.014 ms	6.555 ms	5.710 ms
FileInputStream	1,000,000	225.184 ms	263.308 ms	284.963 ms	257.818 ms
BufferedInputStream	1,000,000	3.857 ms	5.857 ms	4.503 ms	4.7239 ms

Prueba con 1,000,000 de elementos

Flujo utilizado	**Tamaño (elementos)**	**Ejec. 1 (ms)**	**Ejec. 2 (ms)**	**Ejec. 3 (ms)**	**Promedio (ms)**
FileWriter	100,000	480.889 ms	405.346 ms	556.189 ms	480.808 ms
BufferedWriter	100,000	170.902 ms	257.393 ms	203.318 ms	210.538 ms
FileReader	100,000	1535.359 ms	882.256 ms	898.958 ms	1105.524 ms

BufferedReader	100,000	193.348 ms	92.840 ms	91.050 ms	125.746 ms
FileOutputStream	1,000,000	3621.582 ms	2767.678 ms	2930.488 ms	3106.583 ms
BufferedOutputStream	1,000,000	25.939 ms	24.081 ms	21.633 ms	23.884 ms
FileInputStream	1,000,000	1567.945 ms	1512.200 ms	1572.523 ms	1550.889 ms
BufferedInputStream	1,000,000	21.903 ms	19.372 ms	19.671 ms	20.315 ms

Prueba con 10,000,000 de elementos

Flujo utilizado	**Tamaño (elementos)**	**Ejec. 1 (ms)**	**Ejec. 2 (ms)**	**Ejec. 3 (ms)**	**Promedio (ms)**
FileWriter	100,000	2659.059 ms	4068.985 ms	2618.785 ms	3115.610 ms
BufferedWriter	100,000	1061.612 ms	5479.399 ms	1430.286 ms	2657.099 ms
FileReader	100,000	6078.135 ms	18414.472 ms	6194.304 ms	10228.970 ms
BufferedReader	100,000	835.573 ms	5462.712 ms	1157.885 ms	10228.970 ms
FileOutputStream	1,000,000	26133.161 ms	99885.619 ms	28471.650 ms	51496.810 ms
BufferedOutputStream	1,000,000	204.403 ms	446.567 ms	331.984 ms	327.651 ms
FileInputStream	1,000,000	16348.034 ms	75451.456 ms	17832.709 ms	36544.066 ms
BufferedInputStream	1,000,000	161.972 ms	607.754 ms	192.710 ms	320.812 ms

Análisis y preguntas

1. ¿Qué diferencia observaste entre usar flujos con buffer y sin buffer? ¿Por qué ocurre?

Los que tenían buffer fueron más rápidos, sobre todo con los archivos binarios. Pasa porque el buffer permite trabajar con varios datos a la vez y no estar accediendo al archivo tantas veces.

2. ¿Cómo varía el tiempo al aumentar n? ¿La relación es lineal? ¿Se comporta igual en lectura y escritura?

Al aumentar los datos, también aumentaron los tiempos, aunque no siempre de la misma manera. En algunas ejecuciones tardó mucho más que en otras y tampoco fueron iguales los tiempos de lectura y escritura.

3. ¿Existe diferencia entre trabajar con texto y con bytes? ¿A qué se debe?

Sí hubo diferencias. Por ejemplo, los archivos binarios con buffer fueron bastante rápidos, pero sin buffer tardaron mucho. Esto se debe a que el texto necesita convertir los caracteres, mientras que los binarios trabajan directamente con bytes.

4. ¿Qué tamaño tiene el buffer por defecto? ¿Se puede cambiar?

El tamaño es de 8192 bytes para los flujos binarios y 8192 caracteres para los de texto. Sí se puede cambiar indicando otro tamaño al crear el buffer.

5. ¿Por qué BufferedReader tiene readLine () y BufferedInputStream no?

Porque BufferedReader trabaja con texto y puede leer líneas completas. En cambio, BufferedInputStream trabaja con bytes, por lo que no reconoce las líneas de la misma manera.

Conclusión

Se observó como el usar buffer hace bastante diferencia al leer y escribir archivos. También que mientras más datos utilizábamos, más tardaban las operaciones, aunque algunos tiempos variaron bastante. Por lo que se pudo comprender mejor para qué sirve el buffer y por qué conviene utilizarlo cuando trabajamos con muchos datos.